

In-Depth Analysis of Homomorphic Encryption Libraries

A measured comparison of TenSEAL and OpenFHE under CKKS

Itai Zloclzower Jacob Shemesh Tomer Ovadia Yoni Glickstein

Confidential Computing, Ben-Gurion University of the Negev
July 2026

Abstract

We compare two CKKS implementations — TenSEAL (on Microsoft SEAL) and OpenFHE — against a plaintext baseline, across primitive operations, a vector-size sweep, and a multi-stage computation over 1000 encrypted records. Two things distinguish this study from a defaults-versus-defaults comparison. First, we re-run OpenFHE pinned to TenSEAL’s ring dimension and scaling factor, which shows that most of the apparent library gap is a parameter gap: the multiplication penalty falls from $6.89\times$ to $3.16\times$, and the geometric mean across six operations from $6.44\times$ to $3.05\times$ — though decryption remains $7.8\times$ and the matched arm is no longer 128-bit secure. Second, we measure what the IND-CPA^D decryption-noise defence costs, and find that only one of the two libraries offers one at all: TenSEAL’s decryption is bit-deterministic, while OpenFHE randomises by default and supports calibrated flooding for $1.42\times$ on decryption. We also report a cost the literature and our own earlier drafts routinely omit — evaluation keys are 179.7 MB against a 5.3 MB encrypted cohort. This is a course project, and its findings largely reproduce and illustrate known results; its contribution is measurement discipline, including four claims from an earlier draft that did not survive re-examination and are marked as retractions in place.

1 Introduction and Goal

Homomorphic Encryption (HE) allows computation directly on ciphertexts, so a cloud provider can process data it cannot read. This project compares **TenSEAL** and **OpenFHE**, both implementing **CKKS**, against a plaintext baseline.

Research question: what is the practical overhead of computing under HE, and how much of the measured difference between two libraries is a property of the *libraries* rather than of the *parameters* we gave them?

The work has three parts: the cost of individual encrypted operations (§4); a multi-stage computation with realistic arithmetic structure — a synthetic composite score over 1000 encrypted records (§5); and whether CKKS confidentiality suffices for a scenario that releases decrypted results, which ours does (§6).

Four CKKS concepts recur. The **ring dimension** sets the security level and dominates every operation’s cost; the **scaling factor** sets precision; **multiplicative depth** limits sequential multiplications; and **slot packing** lets one ciphertext hold many values so that a single operation advances all of them.

This version follows an adversarial review of an earlier draft. Four of its claims did not survive and are marked as retractions where they occur (§4, §4.2, §4.5, §5.5); §8 records what remains unfixed.

2 Related Work

Cross-library FHE benchmarking is an established genre, and we position this course project inside it rather than claiming novelty.

Library comparisons. Sathya et al. [11] survey HE libraries and their trade-offs. More recently, Faneela et al. [9] benchmark exactly the pair we study — SEAL and OpenFHE — across BGV and CKKS on Linux and Windows, and conclude that OpenFHE is the stronger choice across their settings. A 2025 benchmark study extends the comparison to HELib and Lattigo [10]. That our headline ordering differs from Faneela et al. is itself informative and is the subject of §4.5: library rankings in this area are highly sensitive to the parameters each library is given.

Fair comparison is a known problem, not one we discovered. Takeshita et al.’s *HEProfiler* [8] profiles four CKKS implementations and explicitly “devises methods for fair comparisons of these libraries, even with their widely varied development strategies and library architectures” — the same difficulty our matched-parameter arm addresses, treated far more thoroughly. Viand et al.’s SoK [7] surveys FHE compilers and the comparability problem across backends, and the HEBench framework standardises workloads across libraries. Our §4.5 is a small, single-operating-point instance of a concern these works already articulate.

Scheme and implementation. CKKS is due to Cheon et al. [1]; both libraries implement its full-RNS variant [2], where the modulus-chain and rescaling behaviour we observe lives. The libraries are documented by Al Badawi et al. [3] and Benaissa et al. [4]. OpenFHE’s scaling techniques — whose $1.64\times$ cost and $\sim 6\times$ precision effect we measure — are introduced by Kim et al. [5], which supplies the mechanism for a result we

would otherwise have reported as an unexplained tunable. Parameters follow the Homomorphic Encryption Security Standard [6], the source of the 218/438-bit ceilings we use.

Security of approximate FHE. Li and Micciancio [12] defined IND-CPA^D and demonstrated key recovery against CKKS implementations including SEAL. Li et al. [13] then obtained provable security by post-processing decryption with a differential-privacy mechanism calibrated to *worst-case* error growth; their result *reduces* the flooding required, since earlier approaches left CKKS roughly 8–16 bits of precision. That worst-case qualifier is the hinge: Guo et al. [14] break flooding built on non-worst-case estimation, which is what OpenFHE’s empirical estimator provides. Bergamaschi et al. [15] analyse concrete security at reduced noise, and Alexandru et al. [16] propose application-aware parameter selection.

What we add, modestly. Relative to the above: a matched-parameter ablation that separates ring dimension from scaling technique at one operating point, and a measurement of the IND-CPA^D mitigation’s cost placed alongside the performance numbers rather than discussed separately. Neither is a new technique, and the security mechanism is documented by OpenFHE itself. We contribute no framework and no cryptanalysis.

3 Method

3.1 Configurations

	TenSEAL 0.3.15	OpenFHE 1.4.1
Configured by	modulus chain given directly	depth + scaling size; library picks the rest
Parameters	<code>poly_mod=8192, [60,40,40,60], 2⁴⁰</code>	<code>depth=3, scal_mod=50, HEStd_128</code>
Resulting ring dimension	8192	16384

The two libraries are configured through different routes. That is convenient, and it is also a serious comparability problem — addressed in §4.5. All mandatory operations were implemented for both, plus every optional extension: dot product, the score of §5, scaling tests, separately-timed key generation, the matched comparison, and the IND-CPA^D analysis.

3.2 Measurement

Timing an HE operation once is not reproducible. Every timing is a **mean over 1000 repetitions after 50 discarded warm-up calls**, except: key generation uses 50 repetitions, and the matched-parameter and IND-CPA^D experiments use 200. Warm-up matters because the first calls pay one-off costs — library loading, lazy NTT and key-cache construction, first-touch page faults, frequency ramp-up — that are not steady-state.

Intervals are computed under dependence. Consecutive timings share cache, allocator and scheduler state, so they are not independent and $s/\sqrt{n}$ does not apply. On our archived samples the lag-1 autocorrelation reaches **0.95** and the naive interval is **2–6× too narrow**. Every interval here is a non-overlapping batch-means interval (25 batches); the results files record both, plus the inflation factor.

Stationarity uses three diagnostics. A first-half/second-half comparison is a smoke alarm, not a test: it is blind to a monotonic ramp (which halves its apparent size), to a single spike, to disturbances symmetric about the midpoint, and to load present throughout. We add a **Mann–Kendall** trend test and report lag-1 autocorrelation; a measurement is flagged if either fires. A unit test documents the V-shaped case all three still miss. This earned its place: our first run, on a shared node at load average 10.4 on 8 cores, showed drifts to **73%**, so everything here was re-measured on an **exclusive node**, single-threaded.

Other precautions. `tracemalloc` distorts short timings, so memory is a separate pass; GC is disabled inside the timed loop as `timeit` does; each benchmark asserts decrypted results against the plaintext reference before reporting timings — a crash detector for level exhaustion, deliberately loose, not a precision guarantee. 47 unit tests cover the statistics, the drift blind spots, the score, and the padding invariant.

Caveat on the plaintext denominators. The baseline is pure Python, timed per call at 0.3–1.0 µs against ~130 ns of instrumentation overhead. Every “ $N\times$ slower” figure is therefore *relative to a pure-Python reference* and an upper bound on the ratio against optimised plaintext. We added neither batched timing nor a NumPy baseline.

Environment. Exclusive SLURM node, Intel Xeon E5-2680 v3 @ 2.50 GHz, `OMP_NUM_THREADS=1`, Rocky Linux 9.7, Python 3.8.20. Versions, git commit and node are recorded in every results file. Single-threading is deliberate — thread count is not a library property.

4 Primitive Operations

Input [1.0, 2.0, 3.0, 4.0, 5.0]. “infl.” is how much wider each interval is than the independence formula an earlier draft used.

Operation	Plain (ms)	TenSEAL	CI $\pm$	infl.	OpenFHE	CI $\pm$	infl.	TS/OF vs plain
Encrypt	—	5.4914	0.045	4.6 $\times$	15.1101	0.019	2.1 $\times$	—
Add	0.000629	0.0961	0.000	1.4 $\times$	0.3869	0.000	1.2 $\times$	153 $\times$ / 615 $\times$
Multiply	0.000550	4.7413	0.063	5.8 $\times$	32.4827	0.033	1.6 $\times$	8,616 $\times$ / 59,025 $\times$
Sum	0.000320	10.5082	0.013	2.0 $\times$	76.1546	0.083	2.2 $\times$	32,832 $\times$ / 237,943 $\times$
Average	0.000349	11.7961	0.157	6.0 $\times$	81.3563	0.308	4.5 $\times$	33,794 $\times$ / 233,072 $\times$
Dot Product	0.000993	11.0764	0.014	2.3 $\times$	96.6155	0.066	1.5 $\times$	11,152 $\times$ / 97,272 $\times$
Decrypt	—	1.4029	0.011	4.8 $\times$	22.6702	0.053	2.9 $\times$	—
Total, 5 comparable ops	0.002841	38.2180			286.9961			13,450$\times$ / 101,003$\times$

Per-operation overhead varies by over two orders of magnitude — 153 $\times$ for addition against 32,832 $\times$ for summation — so no single “ $N\times$ slower” figure describes HE. Addition is nearly free; rotation-heavy operations each need a $\log_2(\text{batch size})$ sequence of rotate-and-add steps, every one a key switch, costing 100–120 $\times$ an addition. TenSEAL leads on every operation, but §4.5 shows most of that is parameters.

Two corrections to these totals. The total now sums the **same five operations** in all three columns; an earlier draft divided a seven-operation encrypted total (including encrypt and decrypt, which plaintext lacks) by a five-operation plaintext one and called it a slowdown, inflating the figures to 15,876 $\times$ and 114,299 $\times$. The seven-operation session costs (45.11 / 324.78 ms) are a deployment quantity, reported separately.

More consequentially: an earlier draft argued that 1000 repetitions driving every interval below a few percent made the measurements trustworthy. Our own samples disprove that twice. An interval measures whether samples agree *with each other* assuming independence — the contaminated run had narrow intervals *and* 73% drift. And the samples are not independent even on a quiet machine: lag-1 autocorrelation reaches 0.95, so the naive interval was too narrow by up to 6 $\times$. **Precision is not accuracy**, and this report made the error it was warning about.

4.1 Key Generation

One-off setup, 50 repetitions: **TenSEAL 216.180 ms** (± 0.281), **OpenFHE 208.308 ms** (± 0.206). The only measurement where OpenFHE wins, and the two are within 4%. Setup dominates short sessions: TenSEAL’s key generation costs nearly five times its entire 45 ms benchmark of encrypted operations.

4.2 Memory, and the Object We Forgot to Measure

An earlier draft presented `tracemalloc` figures — all under 1 KB — as memory usage. **That was misleading.** `tracemalloc` sees Python allocations only; both libraries hold coefficients in native heaps, so those were wrapper-object sizes.

Measure	TenSEAL	OpenFHE
<code>tracemalloc</code> , encrypt	0.59 KB	0.42 KB
Resident per ciphertext	359,465 B	1,258,455 B
Serialized	334,428 B	1,312,467 B

That is **596 $\times$** larger for TenSEAL and 2,947 $\times$ for OpenFHE — two to four orders of magnitude, not the six a previous draft claimed (a dropped kilobyte-to-byte conversion, in the paragraph correcting a units error). Resident and serialized agree within $\sim 7\%$. The two libraries are not size-comparable here: different ring dimension and tower count, and SEAL compresses its serialisation where our OpenFHE path does not.

What actually crosses the network. That paragraph led an earlier draft to conclude ciphertext expansion “is what crosses the network.” Measuring the overlooked object reverses it — before the cloud can compute at all it needs the **evaluation keys**:

At the deployed 1000-patient configuration	TenSEAL	OpenFHE
Encrypted cohort (5 features)	5.27 MB	7.87 MB
Evaluation keys	179.7 MB	69.2 MB
Keys $\div$ cohort	34$\times$	8.8$\times$

They are one-off per context, which is why they are easy to omit — and why omitting them misleads any deployment that does not amortise a context over very many queries. Of everything in this revision, this is the correction we would most want carried forward.

4.3 Approximation Error

Maximum absolute error, TenSEAL vs OpenFHE: Add 1.57×10^{-9} vs 4.09×10^{-14} ; Multiply 3.35×10^{-6} vs 1.96×10^{-12} ; Sum 4.60×10^{-7} vs 7.11×10^{-15} ; Average 3.10×10^{-7} vs 5.20×10^{-14} ; Dot 5.19×10^{-6} vs 3.27×10^{-13} . OpenFHE is 10^4 – 10^7 times more precise in this configuration; multiplication dominates the error in both, as expected. All errors are negligible for aggregates meaningful to a few decimal places. The *cause* is not what we assumed — see §4.5.

4.4 Scaling with Vector Size

End-to-end pipeline; context and keys built once per size and excluded.

Size	Plain (ms)	TenSEAL	OpenFHE	TS vs pt	OF vs pt	OF/TS
5	0.001309	37.16	301.55	28,389×	230,368×	8.1×
10	0.001683	51.13	347.45	30,383×	206,446×	6.8×
50	0.004772	98.95	451.32	20,735×	94,576×	4.6×
100	0.008711	119.99	501.16	13,774×	57,532×	4.2×
500	0.044284	271.19	607.03	6,124×	13,708×	2.2×

Over a $100\times$ data increase, plaintext grows $33.8\times$ but **TenSEAL only $7.3\times$ and OpenFHE $2.0\times$** ; per-element cost falls $13.7\times$ and $49.7\times$. The mechanism is slot packing — cost is set by ring dimension, not by how many slots hold data, and OpenFHE held ring 16384 throughout. Only the rotation count, $\log_2(\text{batch size})$, grows.

Two caveats. The plaintext baseline is *itself* sublinear ($n^{0.77}$), because fixed per-call interpreter overhead amortises the same way — so this row is not independent evidence for the CKKS mechanism. And the $n=5$ and $n=10$ plaintext points are both flagged by the drift diagnostics (-5.5% and -10.0% , relative SD above 50%), so the largest overhead figures rest on the least stable denominators in the study.

4.5 Matched-Parameter Comparison

“TenSEAL is faster, OpenFHE is more precise” is not supportable: OpenFHE was doing twice the polynomial work per operation and keeping ten more bits per value. `SetRingDim` (with `HEStd_NotSet`) pins it to 8192 at 2^{40} , matching TenSEAL. We also swept the *scaling technique*, which an earlier draft had silently confounded with the parameter change.

Arm	Encr	Add	Mult	Sum	Dot	Decr	geo.
library-chosen (16384, 2^{50} , FLEXIBLEAUTOEXT)	2.52	4.02	6.89	7.24	8.70	16.28	6.44
matched, same technique (FLEXIBLEAUTOEXT)	1.20	2.12	3.16	3.25	3.97	7.83	3.05
matched, FIXEDMANUAL	1.02	1.77	1.93	2.41	3.09	6.80	2.37
matched, FIXEDAUTO / FLEXIBLEAUTO	1.02	1.75	1.93	2.40	3.09	6.82	2.37

Ratio to TenSEAL, all six operations. An earlier draft showed only the two that converge best.

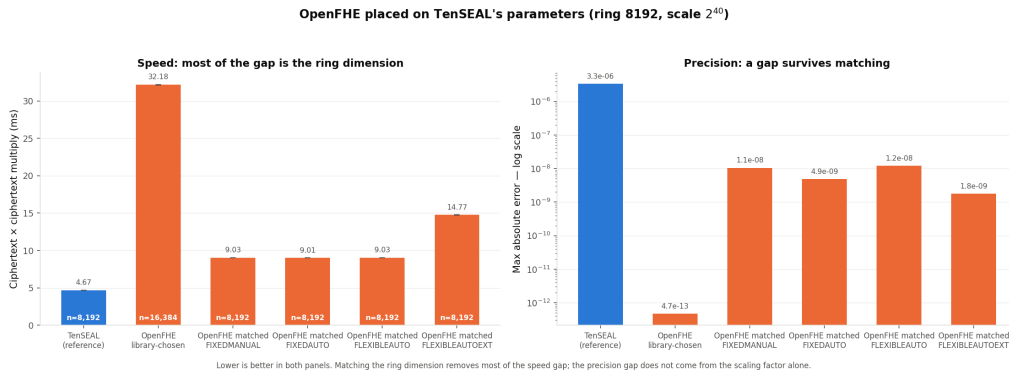


Figure 1: OpenFHE placed on TenSEAL’s parameters. Lower is better in both panels.

- Two variables, separated.** An earlier draft reported “ $6.89\times \rightarrow 1.93\times$ once parameters match” and credited ring dimension alone — while also changing the scaling technique, which the same section showed is worth $1.64\times$. Holding technique fixed: ring 16384 $\rightarrow$ 8192 gives **$2.18\times$** ; FLEXIBLEAUTOEXT $\rightarrow$ FIXEDMANUAL at ring 8192 gives **$1.64\times$** ; product $3.56\times$, exactly the observed closure. The honest parameter-matched figure is **$3.16\times$** .

2. **The gap narrows everywhere but closes nowhere.** Geometric mean falls $6.44\times \rightarrow 3.05\times$. Encryption converges to $1.20\times$; **decryption stays $7.8\times$** — the largest residual, on the operation §6 makes security-critical, and one the earlier table omitted. The summary is a range: **1.2–7.8× by operation**.
3. **Precision is not explained by scale alone.** At identical 2^{40} OpenFHE stays $275\text{--}1,890\times$ more precise. Calling that an *implementation* difference is too strong: the arms also differ in key-switching auxiliary budget and batch size, neither varied, and technique alone moves precision $\sim 6\times$. The mechanism is not established.
4. **The matched arm is not 128-bit secure — a retraction.** An earlier draft claimed it was, comparing OpenFHE’s reported modulus (181–200 bits) against SEAL’s 218-bit ceiling. Different quantities: SEAL’s bound includes its special prime, OpenFHE’s figure omits its hybrid auxiliaries. The decisive test is to ask the library — at HESd_128_classic with these parameters **OpenFHE selects ring 16384 and refuses 8192**, for all four techniques. The experiment now records that refusal. Parameter parity cost security parity.

Still a **trade-off curve, not a ranking** — but weaker than first claimed. This also offers a reading of why Faneela et al. [9] rank OpenFHE above SEAL where we rank TenSEAL above OpenFHE: at defaults the two libraries are not running comparable parameters, and the ordering is a property of the configuration route as much as of the code.

5 Applied Use Case: An Encrypted Composite Score

Disclaimer. The score below is a **synthetic construct for demonstration only**. It is **not** a validated clinical model, is not derived from medical literature, and must not be read as an assessment of patient risk. The cohort is synthetic. Its purpose is to give the benchmark a computation with realistic *structure* — several inputs, mixed arithmetic, genuine multiplicative depth.

5.1 Threat Model, and Why This Workload Is a Proxy

Earlier drafts described the scenario without naming an adversary. **Assets:** individual feature values; the released cohort mean; arguably the formula. **Adversary:** a semi-honest cloud that follows the protocol but inspects everything — ciphertexts, circuit, timings, released outputs. **Trust:** the hospital holds plaintext and the secret key.

What this does not protect. The circuit is public to the cloud (weights, structure and ranges are plaintext constants it evaluates). Cohort size and query pattern leak. The *released aggregate* is a disclosure channel HE does not touch — output privacy, via differential privacy over the decrypted value, is a different problem from IND-CPA^D and we address neither. And there is **no integrity**: a malicious or buggy cloud can return a ciphertext decrypting to anything, undetected. Our accuracy tables check against a plaintext reference the hospital computed itself, which a real deployment could not do.

Why this workload is a proxy. We compare HE against plaintext only, which answers “what does HE cost”, not “should you use HE”. The real alternatives — TEEs, secure multi-party computation, differential privacy alone, or not outsourcing — are outside our scope and we benchmark none. And §5.5 measures the plaintext score for all 1000 records at 0.075 ms on 40 KB of data: **no one would outsource that**, and the client spends more CPU encrypting and decrypting than computing the score itself. The scenario earns its place because its *arithmetic structure* matches workloads where outsourcing is justified.

5.2 The Score

Each feature is min-max normalised against a *public* reference range — age 18–90, systolic BP 90–180, BMI 18–40, cholesterol 120–280, glucose 70–200:

$$\text{score} = 100 \cdot (0.15a + 0.20b + 0.15m + 0.15c + 0.15g + 0.10bg + 0.05mc + 0.05b^2)$$

where each symbol is the normalised feature. The eight weights sum to 1.0. The first five terms need only multiplication by *public constants*. The last three make this a real HE benchmark: two **interaction terms** and one **quadratic term**, each requiring **ciphertext** $\times$ **ciphertext** multiplication, which consumes depth and forces relinearization. The weights were chosen so the terms sum to 1.0 and so that the last three force ciphertext-by-ciphertext products; they are not fitted to any outcome.

Ranges are public deliberately: deriving min/max from the data would need ciphertext comparison, which CKKS does not support natively. **Cohort:** 1000 records, fixed seed; age uniform, the rest normal-then-clipped, giving scores of mean 35.63, SD 8.45, range 9.36–68.90.

5.3 Encrypted Implementation

Packing: one feature per ciphertext, one slot per record — five ciphertexts of 1000 values, so one multiplication advances all 1000. **Padding:** vectors are padded to 1024 slots (OpenFHE requires a power of two) with *each feature’s range minimum*, so they normalise to 0 and drop out of every term and the cohort sum. Padding

with zeros instead gives those slots a nonzero normalised value of $-\min/(\max - \min)$ and silently corrupts the mean; a unit test pins this. **Depth:** normalisation (1) + interaction/quadratic (1) + term weights (1) + cohort mean (1) = 4 levels.

5.4 Depth, Not Data Volume, Drives Cost

The §4 parameters **cannot run this circuit**. TenSEAL at $\text{poly}=8192$ fails with **scale out of bounds**, and any longer chain is rejected outright; $\text{poly}=16384$ with 4 levels works. OpenFHE at depth 3 evaluates the score but fails on the cohort mean; depth 4 works at ring 16384; depth 5 works but pushes the ring to 32768. The rejection is not a quirk: SEAL’s published table caps the coefficient modulus at **218 bits for ring 8192** and 438 for 16384, and $[60, 40, 40, 60] = 200$ fits while $[60, 40, 40, 40, 60] = 240$ does not.

So a modest increase in circuit complexity forces new cryptographic parameters, and asking for more depth than needed is an easy and costly mistake — our first implementation requested depth 5, doubling the ring for no benefit. Two caveats we previously omitted: this holds at a *fixed scaling factor* (the constraint is $\text{depth} \times \text{scale}$), and depth 4 is minimal for *our formulation*, not the circuit — folding term weights into the normalisation constants and doing $\times 1/N$ after decryption would give depth 2.

5.5 Results

Stage	Runs at	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)
Encrypt 5 features	Hospital	—	76.784	84.650
Score evaluation	Cloud	0.0593	89.249	163.181
Cohort mean	Cloud	0.0155	46.104	146.054
Decrypt scores	Hospital	—	2.186	17.457
Pipeline total		0.0748	214.323	411.341
Overhead vs plaintext		1×	2,865×	5,499×
Per record		0.000075	0.2143	0.4113
Cloud-side only	Cloud	0.0748	135.353	309.235

Accuracy: maximum absolute error 4.881×10^{-4} (TenSEAL) and 1.976×10^{-10} (OpenFHE) score points; encrypted cohort mean 35.627391 and 35.626948 against a plaintext 35.626948.

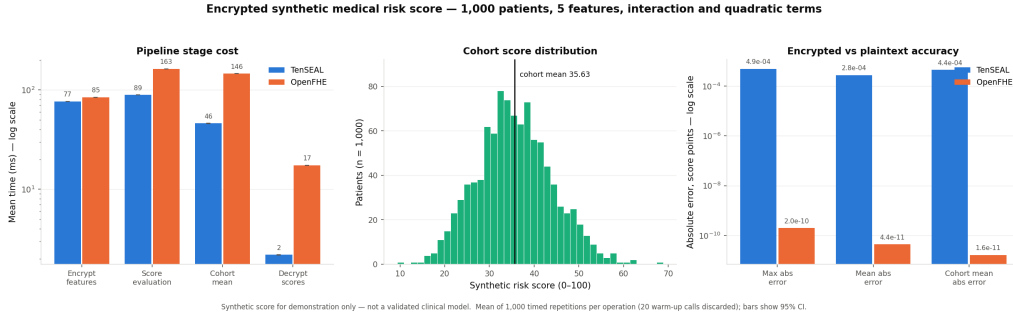


Figure 2: Encrypted score over 1000 records: stage cost, cohort distribution, accuracy.

The overhead figures are a correction. An earlier draft reported 3,614× and 6,936×, using a plaintext denominator of 0.0593 ms — the score stage alone — against an encrypted numerator summing four stages. Worse, the two plaintext stages *overlapped*: the cohort-mean stage recomputed the scores, so it contained the score stage and the two could not be summed either. Stages are now disjoint and the ratios fall ~21%.

The scenario runs in **214 ms** (TenSEAL) or 411 ms (OpenFHE) for 1000 records. Overhead is well below the isolated primitives (2,865× against 13,450×): nothing got faster, the ciphertexts got fuller — though still only one-eighth full, so the amortisation argument is made below saturation.

What the per-record number omits. It says computation is not the bottleneck; it does not say the system is deployable. Excluded: key generation; the **evaluation keys** at 179.7 MB and 69.2 MB (§4.2); transport of 5.3 MB; key custody and rotation. Client and server also share one process holding the secret key, so “the cloud never sees plaintext” is a property of the simulation.

6 Security: Is Encrypting Enough?

CKKS is IND-CPA secure. IND-CPA says nothing about releasing *decryption results* — and our scenario releases them. A CKKS decryption returns the message *plus an error term that depends on the secret key*; an

adversary that obtains approximate decryptions of ciphertexts whose exact plaintexts it can predict can solve for the key [12]. We deliberately do not quote a query count.

6.1 What the Libraries Do, and What the Defence Costs

§6 runs on a **64-record** cohort (batch 64, depth 4, ring 16384, 200 repetitions), not §5’s 1000; the noise estimate is circuit- and data-dependent and does not transfer unchanged. These arms are **not matched to each other** — the criticism §4.5 levels at the performance comparison applies here too.

The observable signature of a decryption-noise defence is **randomization**: plain CKKS decryption is deterministic, so one fixed ciphertext must decrypt identically every time. Over 32 trials: **TenSEAL** — not randomized, spread **exactly 0**; **OpenFHE FIXED_NOISE_DECRYPT** (default) — randomized, 5.71×10^{-11} ; **OpenFHE NOISE_FLOODING_DECRYPT** — randomized, 1.28×10^{-13} .

In the version tested **TenSEAL’s decryption is bit-deterministic**, so the released value carries exactly the key-dependent error the attack exploits, and no setting changes it. OpenFHE randomizes by default — which surprised us.

The table contains an inversion we cannot explain away. The “calibrated” arm adds $\sim 450\times$ less noise than the default it supposedly improves on, and is correspondingly more accurate. The consistent reading is that the default applies a conservative constant while flooding calibrates to this circuit’s measured error, which for so shallow a circuit is smaller. If so, the two-pass arm is not straightforwardly stronger. We did not establish which mode is more secure.

Flooding costs **1.42 $\times$ on decryption and nothing elsewhere** (encrypt, multiply and sum all 1.00 $\times$), plus a full extra pass for estimation.

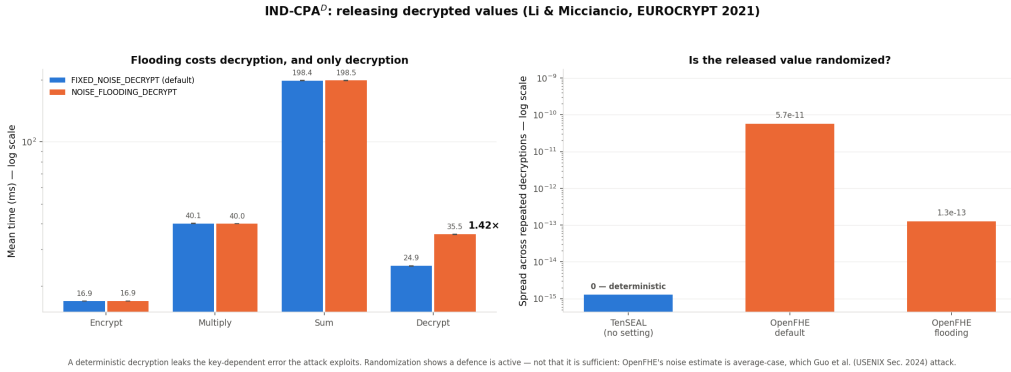


Figure 3: Cost of the flooding defence, and which configurations randomize the released value.

6.2 Why We Do Not Claim This Makes It Secure

OpenFHE estimates circuit noise **empirically**, from precision loss in the imaginary slots of a trial decryption — an **average-case** estimate. Guo et al. [14] show that flooding built on non-worst-case estimation remains vulnerable to key recovery, explicitly including deployments implementing the CRYPTO 2022 bounds. Average-case analysis assumes independent input ciphertexts; our circuit combines correlated ones.

So §6 measures the cost of a partial defence, not the cost of security. Randomized decryption shows a defence is *active*, not *sufficient*; we neither mounted the attack nor verified a bound. The same restraint applies in the other direction, and an earlier draft did not apply it: determinism establishes that TenSEAL offers **no mitigation**, not that our scenario is exploitable.

7 Usability

Aspect	TenSEAL	OpenFHE
Installation	<code>pip install</code> , all platforms	Linux only , one CPython version
API	High-level (<code>enc + enc</code>)	Low-level (<code>cc.EvalAdd(ct, ct)</code>)
Parameter control	Modulus chain explicit	Depth given; ring chosen by library
Scaling technique	Not exposed	Four options, real trade-off (§4.5)
Decryption-noise defence	None	Default fixed noise; opt-in flooding

TenSEAL is far easier for prototyping. OpenFHE exposes controls that matter a great deal (§4.5, §6) and that TenSEAL does not offer at all. One genuine advantage of its style: declaring depth up front makes exceeding it fail immediately, where TenSEAL’s failure surfaces later as **scale out of bounds**.

8 Limitations

Several were identified by adversarial review of an earlier draft and are recorded rather than fixed, because fixing them needs measurement we did not do.

Measurement. (1) **Arms are not interleaved or replicated** — one process, fixed order, so every interval is *within-run* and run-to-run variance (often dominant; [17]) is unmeasured; two runs of the same OpenFHE configuration in our own artifacts differ 10% on encryption. (2) **Plaintext denominators are interpreter measurements** with relative SD above 50%; ratios against them are upper bounds relative to optimised plaintext. (3) **Warm-up is short for TenSEAL**, whose transient runs a few hundred calls, biasing its means slightly high — an asymmetric effect. (4) **The stationarity checks miss disturbances symmetric about the midpoint.** (5) **One node, one CPU generation, single-threaded** — AVX2 but not AVX-512, and the two libraries have different vectorisation backends, so even the ratios may shift on newer hardware.

Cryptographic scope. (6) **The matched arm is not 128-bit secure** (see §4.5). (7) **The two “multiply” operations differ:** TenSEAL relinearises and rescales, OpenFHE defers the rescale, so $3.16\times$ is a lower bound. (8) **Depth 4 is minimal for our formulation, not the circuit.** (9) **“Depth drives ring dimension” holds at fixed scale.** (10) **Ciphertext-size comparison is parameter-determined**, and SEAL compresses where our OpenFHE path does not.

Scenario. (11) Semi-honest adversary only; **no integrity.** (12) **No alternatives benchmarked** — no TEE, MPC or DP baseline. (13) **The per-record figure excludes most of a deployment**, and client and server share one process. (14) **Output privacy and regulation are untreated.** (15) **Determinism is an observational bound** at $n=32$ trials. (16) **The score is synthetic throughout**, with independently drawn features where real measures correlate. (17) **No bootstrapping.**

9 Conclusions

1. **Cost depends on the operation and on how full the ciphertexts are.** Overhead ran from $153\times$ (addition) to $32,832\times$ (summation) on a 5-element vector, falling to $2,865\times$ for the whole pipeline once 1000 values shared each ciphertext — still only one-eighth full. A slowdown quoted without a fill fraction is close to uninterpretable.
2. **Most of the library gap was a parameter gap — but less than we first said.** Matched, and holding the scaling technique fixed, OpenFHE’s multiply penalty falls $6.89\times \rightarrow 3.16\times$, not the $1.93\times$ an earlier draft got by changing two variables at once. Decryption still trails $7.8\times$. And the matched arm is **not** 128-bit secure.
3. **Depth, not data volume, drove cost in the regime measured** — at fixed scaling factor, and only while data fits the available slots.
4. **Rigorous measurement changed the numbers, then changed them again.** Contaminated runs showed 73% drift behind narrow intervals; the intervals were *also* up to $6\times$ too narrow on a quiet machine, because consecutive timings are autocorrelated. Both are the same error — treating an interval as evidence of validity — and this report made it twice.
5. **The largest object crossing the network is the one we did not measure.** Evaluation keys are 179.7 MB against a 5.3 MB encrypted cohort.
6. **Encryption is not the whole security question.** Our scenario releases decrypted aggregates, exactly where CKKS is not IND-CPA^D secure. TenSEAL offers no mitigation; OpenFHE randomises by default and offers calibrated flooding for $1.42\times$ on decryption — but this is a partial defence at best, and we could not explain why the calibrated arm adds less noise than the default.
7. **Who should act.** Engineers choosing a CKKS library for a pipeline that *releases decrypted values* should weigh §6 above §4. Anyone quoting HE benchmarks should state fill fraction and parameters, which move headlines by an order of magnitude — and, as §4.5 suggests, can reverse a library ranking.

References

- [1] J. H. Cheon, A. Kim, M. Kim, Y. Song. Homomorphic encryption for arithmetic of approximate numbers. *ASIACRYPT*, 2017.
- [2] J. H. Cheon, K. Han, A. Kim, M. Kim, Y. Song. A full RNS variant of approximate homomorphic encryption. *SAC*, 2018.
- [3] A. Al Badawi et al. OpenFHE: open-source fully homomorphic encryption library. *WAHC*, 2022.
- [4] A. Benaissa, B. Retiat, B. Cebere, A. E. Belfedhal. TenSEAL: a library for encrypted tensor operations. 2021.
- [5] A. Kim, A. Papadimitriou, Y. Polyakov. Approximate homomorphic encryption with reduced approximation error. *CT-RSA*, 2022.
- [6] M. Albrecht et al. Homomorphic Encryption Security Standard. HomomorphicEncryption.org, 2018.

- [7] A. Viand, P. Jattke, A. Hithnawi. SoK: fully homomorphic encryption compilers. *IEEE S&P*, 2021.
- [8] J. Takeshita, N. Koirala, C. McKechney, T. Jung. HEPProfiler: an in-depth profiler of approximate homomorphic encryption libraries. *J. Cryptographic Engineering*, 2025.
- [9] Faneela, J. Ahmad, B. Ghaleb, S. U. Jan, W. J. Buchanan. Cross-platform benchmarking of the FHE libraries: novel insights into SEAL and OpenFHE. arXiv:2503.11216, 2025.
- [10] Performance analysis of leading homomorphic encryption libraries: a benchmark study of SEAL, HElib, OpenFHE and Lattigo. *ICCSAIDE*, 2025.
- [11] S. S. Sathya, P. Vepakomma, R. Raskar, R. Ramachandra, S. Bhattacharya. A review of homomorphic encryption libraries for secure computation. arXiv:1812.02428, 2018.
- [12] B. Li, D. Micciancio. On the security of homomorphic encryption on approximate numbers. *EUROCRYPT*, 2021.
- [13] B. Li, D. Micciancio, M. Schultz, J. Sorrell. Securing approximate homomorphic encryption using differential privacy. *CRYPTO*, 2022.
- [14] Q. Guo, D. Nabokov, E. Suvanto, T. Johansson. Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. *USENIX Security*, 2024.
- [15] F. Bergamaschi et al. Revisiting the security of approximate FHE with noise-flooding countermeasures. *PKC*, 2025.
- [16] A. Alexandru, A. Al Badawi, D. Micciancio, Y. Polyakov. Application-aware approximate homomorphic encryption. ePrint 2024/203.
- [17] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney. Producing wrong data without doing anything obviously wrong! *ASPLOS*, 2009.

Code, notebook and charts accompany this report; every figure is regenerable with `./run_all_benchmarks.sh`. Software measured: TenSEAL 0.3.15, OpenFHE 1.4.1.